Technical Report: Slime Mold Algorithm Bio-Inspired Simulation in Ad-Natura

SFWRENG 4G06 - Capstone Design Project

Team #7, Wardens of the Wild
Felix Hurst
Marcos Hernandez-Rivero
BoWen Liu
Andy Liang

March 26, 2026

Contents

1 Revision History **ii**

2 Introduction **1**

3 System Architecture **1**
3.1 Data Structures . . . 1
3.2 Ping-Pong Buffering . . . 1
3.3 Kernel Overview . . . 2

4 Agent Behavior and Chemotaxis **2**
4.1 Sensing . . . 2
4.2 Turning . . . 2
4.3 Movement and Boundary Handling . . . 3
4.4 Trail Deposition . . . 3
4.5 Behavioral Drift . . . 3

5 Trail Dynamics **4**
5.1 Diffusion . . . 4
5.2 Entropic Decay . . . 4
5.3 Cytoplasmic Streaming (Pulse) . . . 5
5.4 Metabolic Glow . . . 5

6 GPU Noise Implementation **6**

7 Game Integration **6**

8 Design Rationale **7**
8.1 Self-Organization Without Central Control . . . 7
8.2 Externalized Memory . . . 8
8.3 Counter-Algorithmic Design . . . 8
8.4 Nature's Performativity . . . 8
8.5 Non-Human Play . . . 8

9 Technical Challenges and Lessons **9**
9.1 The Reservoir Effect . . . 9
9.2 GPU Noise Precision . . . 9
9.3 RenderTexture Format . . . 9
9.4 Inspector Serialization . . . 9

10 Microphone as Interface **9**
10.1 Parameter Mapping . . . 10

11 Consent and Resistance **10**
11.1 Stress Dynamics . . . 10
11.2 Refractory State . . . 11

12 Conclusion **11**

1 Revision History

Date	Version	Author	Notes
Mar 7, 2026	1.0	Andy Liang	Initial report: architecture, implemented behaviors, design rationale
Mar 21, 2026	1.1	Andy Liang	Added behavioral drift, water consumption, metabolic glow; updated design rationale with Sims and Adamatzky
Mar 26, 2026	2.0	Andy Liang	Final version: microphone input, consent/resistance, light avoidance implemented; TA feedback addressed; trail deposition corrected; Future Work moved after Conclusion

2 Introduction

This report documents the design and implementation of the slime mold simulation system in *Ad-Natura*, a 2D environmental exploration game. The simulation models *Physarum polycephalum*, a single-celled organism capable of decentralized decision-making, adaptive network formation, and environmental sensing—without any central nervous system or governing structure [6, 1].

The system runs entirely on the GPU as a compute shader, driving 1024 autonomous agents that deposit chemical trails, sense their environment, and collectively produce emergent network structures. Beyond serving as a game mechanic, the simulation is designed as a site of inquiry: a counter-algorithmic system [5] that resists optimization in favor of organic unpredictability.

This report covers the technical architecture (Section 3), agent behavior (Section 4), trail dynamics (Section 5), noise generation (Section 6), game integration (Section 7), microphone input (Section 10), consent and resistance (Section 11), design rationale (Section 8), and technical challenges (Section 9).

3 System Architecture

The simulation is implemented as a Unity compute shader (`Slime.compute`) driven by a C# controller (`Slime.cs`). It uses a ping-pong double-buffering scheme and two GPU kernels dispatched each `FixedUpdate` frame.

3.1 Data Structures

Each agent is represented by a minimal struct:

```
struct Agent {
    float2 position;   // world-space position
    float  angle;      // heading in radians
    float4 type;       // species color (unused in current build)
};
```

Agents are stored in a `StructuredBuffer` of 1024 elements, initialized with random positions and angles via a uint-based hash PRNG.

3.2 Ping-Pong Buffering

Two `RenderTexture` buffers (`TrailMap` and `TrailMapProcessed`) alternate roles each frame. The **Update** kernel reads from the current trail map and writes agent deposits to it. The **Postprocess** kernel reads from that same buffer, applies diffusion, decay, and pulse, then writes the result to the other buffer. On the next frame, the roles swap. This prevents read-write hazards on the GPU.

Both trail maps use a floating-point texture format (`RenderTextureFormat.RFloat` for attraction maps, `RenderTextureFormat.ARGBFloat` for trail maps). This is critical: trail

intensity values routinely exceed 1.0, creating a "reservoir effect" where dense trail networks store accumulated chemical signal. Clamping to $[0, 1]$ destroys this behavior entirely.

3.3 Kernel Overview

- **Update kernel** (`[numthreads(16,1,1)]`): One thread per agent. Handles sensing, turning, movement, boundary behavior, and trail deposition. Dispatched as $\lceil 1024/16 \rceil = 64$ thread groups.
- **Postprocess kernel** (`[numthreads(8,8,1)]`): One thread per texel. Handles 3×3 diffusion, entropic decay, cytoplasmic pulse, and metabolic glow. Dispatched over the full trail map resolution.

4 Agent Behavior and Chemotaxis

The simulation models the organism as a population of 1024 *agents*—autonomous computational particles, each representing a small region of the plasmodial network. An agent has a position, a heading angle, and no memory beyond these two values. It cannot communicate with other agents directly; all coordination happens indirectly through chemical trails deposited in the shared environment (stigmergy). Each agent performs a sense-turn-move-deposit cycle inspired by the Jones model of Physarum chemotaxis [1].

4.1 Sensing

The `sense()` function samples a 3×3 neighborhood at a point offset from the agent's position by `sensorOffsetDist` in a given direction. It sums both trail intensity and water attraction values, weighted by `waterWeight`:

$$S(\theta) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} \Big[\text{trail}(p + d_\theta + (i, j)) + w \cdot \text{water}(p + d_\theta + (i, j)) \Big] \tag{1}$$

where p is the agent position, d_θ is the sensor offset vector at angle θ, and w is the water attraction weight.

4.2 Turning

Three sensor readings are taken: forward (S_F), left (S_L, at +`sensorAngleSpacing`), and right (S_R, at −`sensorAngleSpacing`). The turning logic follows:

1. If $S_F > S_L$ and $S_F > S_R$: continue straight.
2. If $S_F < S_L$ and $S_F < S_R$: turn randomly left or right (stochastic tie-breaking prevents deadlock).
3. If $S_L > S_R$: turn left by `turnSpeed` $\cdot \Delta t$.

4. If $S_R > S_L$: turn right by `turnSpeed` $\cdot \Delta t$.

This produces emergent path-following and network formation without any global coordination—each agent responds only to local chemical gradients.

4.3 Movement and Boundary Handling

After sensing and turning, each agent advances along its heading. Speed is a continuous blend between an idle base speed and an attracted speed, governed by local water attraction:

$$v = \mathrm{lerp}\big(\texttt{idleSpeed},\ \texttt{moveSpeed} \cdot \texttt{attractedSpeedMultiplier},\ \mathrm{saturate}(\mathrm{attraction})\big) \quad (2)$$

This produces smooth acceleration toward water rather than a binary fast/slow switch. Agents far from any stimulus still drift slowly, keeping the colony alive even in barren regions.

At boundaries, the behavior depends on whether water is nearby. If the agent hits an edge where the water attraction map reads above a small threshold, it deflects *along* the boundary (horizontal or vertical, chosen randomly) with a random angular spread. This prevents the pile-up that occurs when agents bounce straight back and immediately re-deposit on the same pixel. If no water is sensed at the edge, the agent receives a random heading biased away from the wall, with slight randomness to avoid artificial symmetry. In both cases, the agent's position is clamped to stay within bounds.

4.4 Trail Deposition

Each agent overwrites its current texel with the organism's base color at a fixed intensity:

$$\texttt{TrailMap}[\mathrm{pos}] = \mathbf{c}_{\mathrm{slime}} \cdot \mathrm{trailStr} \quad (3)$$

where trailStr = 1.0 under normal conditions. The deposit is a hard overwrite, not an addition—a single agent on an empty pixel produces the same value as ten agents on the same pixel. Veins emerge not from additive stacking but from *reinforcement*: many agents repeatedly refreshing the same path before evaporation can fade it. Unpopular paths evaporate away; popular paths persist. The `trailWeight` parameter modulates how strongly agents *sense* trails (Section 4), not how much they deposit.

4.5 Behavioral Drift

In a real *Physarum* population, no two regions of the plasmodium behave identically. To model this, each agent's parameters are modulated by a per-agent noise seed derived from its index:

$$p_i = p_{\mathrm{base}} \cdot (1 + m \cdot (2 \cdot \mathrm{hash}(i, t) - 1)) \quad (4)$$

where p_i is the agent's effective parameter value, p_{base} is the shared base value, m is the drift multiplier, and $\mathrm{hash}(i, t)$ is a slowly evolving noise value seeded by the agent's

index. Four parameters are modulated independently: sensor length ($m = 0.7$), turn speed ($m = 0.6$), trail weight ($m = 0.8$), and water attraction strength ($m = 0.9$).

The drift evolves over time (controlled by `driftSpeed`), so an agent that is currently exploratory may later become conservative, and vice versa. The population is simultaneously individual *and* collective—what Sinclair [8] describes as paraconsistent individuality, where an entity can hold contradictory states without resolution. The emergent effect is a simulation that never fully converges to a single stable pattern.

5 Trail Dynamics

The Postprocess kernel transforms the raw trail map each frame through four sequential stages: diffusion, entropic decay, cytoplasmic pulse (with brightness tapering), and metabolic glow.

5.1 Diffusion

A standard 3×3 box blur computes the mean trail value in the neighborhood of each texel. The result is linearly interpolated with the original value:

$$T'(x, y) = \text{lerp}\Big(T(x, y),\ \frac{1}{9} \sum_{i,j \in \{-1,0,1\}} T(x + i,\ y + j),\ \texttt{diffuseSpeed} \cdot \Delta t\Big) \tag{5}$$

This simulates the physical diffusion of chemical signal through the environment, smoothing the trail network over time.

5.2 Entropic Decay

Rather than applying a uniform evaporation rate, the system modulates decay with a two-octave noise field that varies in both space and time:

$$\text{noise}(x, y, t) = \texttt{organicNoise}\Big(\frac{x \cdot s_e}{W},\ \frac{y \cdot s_e}{H},\ t \cdot v_e\Big) \tag{6}$$

$$e(x, y, t) = \texttt{evaporateSpeed} \cdot \Delta t \cdot \big(1 + \texttt{entropyStrength} \cdot (2 \cdot \text{noise} - 1)\big) \tag{7}$$

$$T''(x, y) = T'(x, y) - e(x, y, t) \tag{8}$$

where s_e is the entropy scale, v_e is the entropy speed, W and H are texture dimensions. The noise field creates regions of faster and slower decay, producing organic, non-uniform trail dissolution. Some areas persist longer while others erode quickly, breaking the visual monotony of uniform evaporation.

This feature is grounded in the observation that real biological decay is never uniform—it is shaped by local moisture, temperature, microbial activity, and countless other environmental variables [5].

5.3 Cytoplasmic Streaming (Pulse)

Real *Physarum* exhibits rhythmic cytoplasmic streaming—the shuttle flow of protoplasm that oscillates through the organism's network, driving nutrient transport and enabling distributed decision-making [1].

The simulation reproduces this as a density-thresholded brightness oscillation:

$$d_n = \text{smoothstep}(0.3,\ 0.6,\ T''(x,y)/T_{\max}) \tag{9}$$

$$f(x,y) = \text{lerp}\big(\texttt{pulseMinFreq},\ \texttt{pulseMaxFreq},\ \text{water}(x,y)\big) \tag{10}$$

$$\phi(x,y) = (x+y) \cdot \texttt{pulseWaveScale} \tag{11}$$

$$\text{pulse}(x,y,t) = d_n \cdot \texttt{pulseAmplitude} \cdot \sin(2\pi f t + \phi) \tag{12}$$

The final trail output is:

$$T_{\text{out}}(x,y) = \max\big(T''(x,y) + \text{pulse}(x,y,t),\ 0\big) \tag{13}$$

Key design decisions:

- **Density threshold**: The smoothstep activation ensures only established trail networks pulse, not sparse background noise. This mirrors real Physarum, where streaming occurs in mature veins.
- **Water-driven frequency**: Pulse frequency increases near water sources, simulating heightened metabolic activity near nutrients.
- **Spatial phase offset**: The $(x+y)$ phase term creates the illusion of wave propagation across the network, rather than uniform blinking.
- **No clamping**: The output uses $\max(\cdot, 0)$ rather than $\text{clamp}(0,1)$, preserving the reservoir effect of accumulated chemical signal.
- **Brightness taper**: Because the pulse operates within a ping-pong buffer, its multiplicative boost can ratchet brightness upward across frames in dense clusters. A soft taper scales the pulse contribution down as brightness approaches 2.0: $\text{taper} = \text{saturate}(2.0 - \|\mathbf{T}\|)$. This preserves the glow in dense networks while preventing blowout.

5.4 Metabolic Glow

When the slime mold consumes water from the environment (Section 7), a `metabolicGlow` value is tracked on the CPU and passed to the shader. The Postprocess kernel applies this as an additive brightness boost, localized to regions where trails and water overlap:

$$T_{\text{out}}(x,y) = T_{\text{pulse}}(x,y) + \mathbf{c}_{\text{slime}} \cdot g \cdot 0.05 \cdot d_n \cdot w(x,y) \tag{14}$$

where g is the current metabolic glow intensity, $\mathbf{c}_{\text{slime}}$ is the organism's base color, d_n is the same density threshold used by the pulse, and $w(x, y)$ is the local water attraction value. This localizes the glow to feeding sites rather than illuminating the entire network—the bloom appears where the organism meets the water, not everywhere it has been. The glow is strictly additive, not multiplicative—a critical design decision. Because the Postprocess kernel operates within a ping-pong buffer, any multiplicative amplification ($T \leftarrow T \cdot (1 + g)$) would compound exponentially across frames, producing catastrophic feedback. Additive glow reaches a natural equilibrium with evaporation, producing a visible but stable brightness increase that decays smoothly as the glow value fades on the CPU side.

The effect is a brief luminous bloom when the organism feeds—a visible metabolic event that makes the act of consumption perceptible.

6 GPU Noise Implementation

GPU-based noise requires careful implementation. Standard approaches using sin()-based hash functions suffer from precision artifacts on many GPU architectures. The system uses an integer hash approach:

```
float hashNoise(float2 p) {
    uint2 i = uint2(abs(p));
    uint  h = i.x * 1597334677u ^ i.y * 3812015801u;
    h = h * h;
    return float(h) / 4294967295.0;
}
```

This provides well-distributed pseudorandom values without relying on trigonometric functions. A `valueNoise()` function interpolates between hash corners using smoothstep, and `organicNoise()` layers two octaves with halved amplitude:

$$\text{organicNoise}(x, y, t) = \text{valueNoise}(x, y, t) + 0.5 \cdot \text{valueNoise}(2x, 2y, 2t) \tag{15}$$

The result is normalized to $[0, 1]$ by dividing by 1.5. This two-octave structure adds fine detail to the noise field without the computational cost of full fractal Brownian motion, which is unnecessary for the visual effect.

7 Game Integration

While the simulation is designed as a self-contained system, it integrates with the broader game through several mechanics:

- **Water Attraction**: A water attraction map (`WaterAttractionMap`) is generated by the game's water simulation system and passed to the compute shader. Agents preferentially move toward water, deposit stronger trails near it, and pulse at higher frequencies in its proximity.

- **Decomposition**: The slime mold participates in the game's decomposition system, breaking down organic obstacles in the environment. Trail density in an area contributes to the decomposition rate of nearby objects.

- **Water Consumption**: The organism actively consumes water from the game's cellular liquid simulation. The C# controller periodically samples trail density across the texture, identifies regions where both slime trails and water are present, and calls `RemoveWater()` on the liquid simulation to drain a cluster of neighboring grid cells. A `metabolicGlow` value accumulates during consumption and decays over time, driving the visual glow in the shader (Section 5.4). This creates a reciprocal material relationship: the water shapes the organism's growth, and the organism depletes the water. Neither remains unchanged by the encounter—what Barad [4] terms intra-action, where entities co-constitute through mutual transformation rather than one-directional influence.

- **Light Avoidance**: Real *Physarum* is negatively phototactic. The simulation encodes this via the R channel of the `WaterAttractionMap`, which stores a light fear value. Agents in lit areas blend between trail-following chemotaxis and light-fleeing behavior: fear dominates but trail signal still contributes, so the organism *resists* light rather than losing all self-organization. This preserves emergent network behavior under environmental pressure.

- **Hazard System**: Calamity objects (encoded in a separate `HazardMap`) corrode trails, slow agents, and kill agents on direct contact. Dead agents leave a colored death mark at their last position and respawn at the colony center. The stain falloff is pre-computed on the CPU to avoid expensive per-pixel GPU neighbor searches.

The `SlimeMoldManager` class coordinates between the simulation and other game systems, providing a public API (`SetAttractionMaps`, `GetTrailTexture`, `GetWorldBounds`) for other scripts to interact with the slime mold.

8 Design Rationale

The design of this system is informed by both biological research and critical theory. The artist's role here is not that of a central author imposing form onto matter, but a designer of conditions under which matter performs [2]. Several deliberate choices distinguish it from conventional algorithmic implementations.

8.1 Self-Organization Without Central Control

Evelyn Fox Keller's foundational work on slime mold aggregation [6] demonstrated that scientists historically assumed *Dictyostelium* required a "pacemaker" cell to coordinate collective behavior. Keller showed that aggregation is fully self-organizing: no leader, no central governor.

This principle is encoded in the simulation's architecture. All 1024 agents follow identical local rules—sense, turn, move, deposit. Network formation, trail reinforcement, and path

optimization emerge entirely from these local interactions. There is no global pathfinding, no master controller, no optimization objective.

8.2 Externalized Memory

The trail map functions as a form of externalized spatial memory. As Sims [3] observes, slime mold does not store memory internally; it remembers through the chemical traces it leaves in its environment. In the simulation, the trail network is simultaneously the organism's history, its communication medium, and the substrate for collective decision-making. Entropic decay (Section 5.2) ensures this memory is impermanent and shaped by environmental noise—not a perfect record but an organic, degrading trace.

8.3 Counter-Algorithmic Design

Bunton [5] critiques the Slime Mold Algorithm (SMA) for reducing *Physarum*'s complex, "queer" biology to a metaheuristic optimization tool—stripping away precisely the behaviors that make the organism interesting. The concept of "jamming"—blocking, improvising, knotting—is proposed as an alternative relation to algorithmic systems.

The entropic decay system (Section 5.2) functions as a form of jamming. Rather than optimizing trail networks toward efficiency, the noise-modulated decay introduces unpredictable dissolution. Trails that "should" persist may erode; regions that "should" clear may linger. The system resists convergence toward optimal solutions.

8.4 Nature's Performativity

Barad's theory of agential realism [4] argues that nature is not passive matter awaiting measurement, but actively performs—entities co-constitute through "intra-action" rather than interaction. The slime mold simulation embodies this: the organism and its environment are not separate systems that interact, but a single entangled process. Trail deposition shapes agent movement, which shapes trail deposition. The water map shapes the organism, and in the game's decomposition system, the organism reshapes its environment.

8.5 Non-Human Play

Stone [9] extends the concept of "player" beyond the human, drawing on Bennett's distributive agency to argue that nonhuman entities—bacteria, plants, algorithms—can be understood as players in an assemblage. In *Ad-Natura*, the slime mold is not an object the player controls but a co-player with its own agency: it responds to the environment, makes decisions (through chemotaxis), and produces outcomes the human player cannot fully predict or control.

This framing connects to Ruberg's concept of "after agency" [7]—game systems where the human player's control is deliberately limited. The player can influence the slime mold's environment (e.g., directing water flow), but cannot command the organism directly.

9 Technical Challenges and Lessons

9.1 The Reservoir Effect

The most consequential technical discovery during development was that trail values in floating-point `RenderTexture` buffers routinely exceed 1.0. When multiple agents deposit trails in the same region over several frames, the accumulated value grows well above the $[0, 1]$ range. This "reservoir effect" is essential to the simulation's visual quality: dense networks appear brighter and more saturated, while sparse trails fade naturally.

An early implementation clamped the output to $[0, 1]$, which flattened all dense networks to a uniform maximum brightness and eliminated the visual distinction between mature veins and new trails. The fix was simple but non-obvious: output $\max(\text{value}, 0)$ with no upper clamp.

9.2 GPU Noise Precision

Initial noise implementations using sin()-based hash functions produced visible banding artifacts on certain GPU architectures. The integer hash approach (Section 6) resolved this entirely, at no performance cost.

9.3 RenderTexture Format

The default water attraction map must use `RenderTextureFormat.RFloat`, not `ARGB32`. Using an integer format caused the attraction values to be silently quantized, producing step-like artifacts in agent movement near water boundaries.

9.4 Inspector Serialization

Unity's Inspector serializes field values independently of code defaults. When default parameter values were changed in the C# script, existing scene references retained their old serialized values. This caused confusion during development, as code changes appeared to have no effect until the component was reset manually.

10 Microphone as Interface

A standalone `MicrophoneInput` component captures audio from the system's default recording device via Unity's `Microphone` API. Three features are extracted each frame:

- **Amplitude**: RMS energy of the most recent 1024 samples, scaled by a configurable gain and smoothed via exponential lerp.
- **Spectral centroid**: The energy-weighted mean frequency bin, computed from Unity's `GetSpectrumData` FFT (Blackman-Harris window). Normalized to $[0, 1]$, where 0 is low-pitched and 1 is high-pitched.

- **Onset**: The positive difference in amplitude between consecutive frames, thresholded and scaled to detect sudden sound events.

A hidden `AudioSource` (muted, zero volume) plays the microphone clip to provide Unity's FFT pipeline with data. The source is muted to prevent audible feedback; `GetSpectrumData` still functions when muted.

10.1 Parameter Mapping

The C# controller reads mic values into local copies of simulation parameters each frame, leaving Inspector-serialized values untouched. A master `micInfluence` slider scales all effects:

- **Silence** (amplitude ≈ 0): Pulse frequency lifts gently. The organism notices the player's quiet presence.
- **Loud sound**: Evaporation rate increases (trails thin), turn speed increases (agents agitate). The network becomes restless.
- **High pitch**: Sensor range contracts, turn speed increases further. The organism becomes nervous, myopic.
- **Low pitch**: Diffusion rate increases. The trail network softens and spreads.
- **Sudden onset**: Trail sensing weight spikes briefly—a short-term memory of the sound event.

Turn speed modulation from amplitude and pitch are combined additively (not multiplicatively) to prevent compounding that spirals agents into dense clusters. Sensor shrink is floored at 50% of base length to ensure agents always retain enough sensing range to disperse.

11 Consent and Resistance

The organism can refuse the player's input. A stress accumulator on the CPU tracks how long the microphone amplitude exceeds a configurable threshold. When stress crosses a second threshold, the organism enters a *refractory state*—a timed withdrawal during which it becomes sluggish, dim, and unresponsive.

11.1 Stress Dynamics

$$\sigma_{t+1} = \begin{cases} \sigma_t + (\text{amp} - \theta_s) \cdot r_a \cdot \Delta t & \text{if amp} > \theta_s \\ \sigma_t - r_d \cdot \Delta t & \text{otherwise} \end{cases} \tag{16}$$

where σ is the stress level, θ_s is the stress threshold, r_a is the accumulation rate, and r_d is the recovery rate. The transition into withdrawal is not instantaneous: as stress builds, the organism's responsiveness to mic input fades proportionally (responsiveness $= 1 - 0.7\sigma$), so the player can observe the organism becoming less reactive before it shuts down entirely.

11.2 Refractory State

When $\sigma \geq \theta_r$ (the refractory threshold), the organism enters withdrawal. A smoothed refractory signal ramps up gradually (preventing a harsh visual snap) and is passed to the GPU, where it dampens:

- **Behavioral drift**: individuality collapses, the collective becomes uniform ($\times 0.9$ reduction).
- **Speed**: the organism becomes sluggish ($\times 0.35$ reduction).
- **Trail deposit**: trails dim ($\times 0.2$ reduction).
- **Pulse amplitude**: breathing shallows ($\times 0.4$ reduction).

Recovery requires silence. If the microphone amplitude remains above the stress threshold during withdrawal, the recovery timer pauses—the organism cannot heal while still being assaulted. This is a deliberate design choice grounded in Ruberg's "after agency" [7]: the player's control is not removed by a game mechanic but rendered meaningless by the organism's refusal.

12 Conclusion

The slime mold simulation in *Ad-Natura* is a GPU-accelerated, agent-based model of *Physarum polycephalum* that prioritizes biological plausibility and emergent complexity over algorithmic efficiency. Through entropic decay, cytoplasmic pulse, behavioral drift, water consumption, metabolic glow, real-time microphone input, and consent-based withdrawal, the system produces organic, unpredictable behavior from simple local rules—while engaging in reciprocal exchange with both its digital environment and the player's physical body.

The microphone transforms the simulation from a reactive display into a site of encounter: the player's breath, voice, and silence all register as input, and the organism responds on its own terms—including the capacity to refuse. The design is grounded in critical engagement with biological research and theoretical frameworks that challenge conventional approaches to computational modeling of living systems. Rather than reducing *Physarum* to an optimization heuristic, the system honors the organism's capacity for self-organization, distributed agency, externalized memory, and resistance to centralized control.

References

[1] A. Adamatzky. Slime mould analog models of space exploration. *International Journal of Bifurcation and Chaos*, 24(02):1430006, 2014.

[2] A. Adamatzky and T. Schubert. *Experiencing the Unconventional: Science in Art*. World Scientific, 2016.

[3] B. Sims. *Slime Mould and Philosophy*. Punctum Books, 2021.

[4] K. Barad. Nature's queer performativity. *Qui Parle*, 19(2):121–158, 2011.

[5] K. Bunton. *Jamming Playbook: Re/Decomposing the Slime Mold Algorithm*. MA thesis, Royal Academy of Art, The Hague, 2025.

[6] E. F. Keller. The force of the pacemaker concept in theories of aggregation. *Journal of the History of Biology*, 16(1):39–56, 1983.

[7] B. Ruberg. After agency: Video games and the non-player character. In *The Routledge Companion to Media and Fairy-Tale Cultures*, pages 479–487. Routledge, 2022.

[8] N. W. Sinclair. Exploding individuals: Engaging indigenous logic within science. *Hypatia*, 35(1):58–74, 2020.

[9] K. Stone. What can play: The potential of non-human players. In *Games and Play in the Creative, Media and Cultural Industries*, pages 165–180. Routledge, 2019.